

KOI CONTENT EXTENSION

Test Guide

Ten tests for the content built on the Marketplace KOI integration — with the setup each one needs

For anyone comfortable in the Cortex XSIAM console. No content development required.

Extension 1.3.0 · requires the Marketplace KOI pack · July 2026

CONTENTS

What ships in this extension

A

Parsing & modeling rules

Normalise KOI events and map them to the Cortex Data Model. The Marketplace pack ships none.

B

Alerts dashboard

Ready-made view of KOI alert activity.

C

Alert fields, type & layout

19 KOI fields written onto each alert and shown in a purpose-built layout.

D

Triage & investigation playbooks

6 playbooks: triage, item and device investigation, enrichment, gated response.

E

Hunting playbooks

3 playbooks: MCP server audit, hunt sweep, and its investigation sub-playbook.

F

Script Runner playbooks

5 playbooks plus the KoiScanTracker automation — run KOI scripts fleet-wide.

The integration itself is NOT in this pack — it comes from the Marketplace KOI pack, which must be installed first.

Every test is laid out the same way

PACK CONTENT

The content items that must be on the tenant for this test — playbooks, automations, rules, Lists. Useful if you upload the pack piece by piece rather than in one go.

TENANT SETUP

What you configure yourself: integration instances, an endpoint group, a script in Action Center, the configuration List.

STEPS + WHAT TO EXPECT

What to click, in order, in the Cortex XSIAM console — and what a passing test looks like. Anything to paste is shown in monospace.

Installed the pack, or uploading piece by piece?

Install the whole pack and every playbook, automation, rule and the Koi Script Runner List are already on the tenant — nothing to upload. If you upload selectively instead, the PACK CONTENT column on each test is your upload checklist for that test.

Run them in order the first time

Tests 1 to 3 prove data is arriving and correct. Tests 4 to 7 prove the automation. Tests 8 and 9 prove fleet script execution — the only ones needing the Core REST API integration. Test 10 is the alert layout. Nothing here needs a Cortex XDR integration: XSIAM is the XDR, and its XQL engine is built in.

Each test lists its own prerequisites — set those up first and the test runs straight through.

BEFORE YOU BEGIN

Everything this pack depends on, and where to set it up

What	Where in XSIAM	Needed for
Marketplace KOI pack	Marketplace → search KOI → Install	Everything — it holds the integration
KOI API key	KOI console → Settings → API Access	Tests 1-7, 10
KOI integration instance	Settings → Data Sources → Add → KOI	Tests 1-7, 10
An egress IP KOI accepts	Run the instance on a Cortex engine if needed	Tests 1-7, 10
Core REST API instance	Settings → Data Sources → Add → Core REST API	Tests 8-9 only
KOI script in Action Center	Action Center → Scripts Library → upload	Tests 8-9
An endpoint group	Endpoints → tag agents, then Endpoint Groups → dynamic	Tests 8-9
"Koi Script Runner" JSON List	Settings → Object Setup → Lists	Tests 8-9



Two to set up before anything else. The Marketplace KOI pack must be installed first — this extension ships content only, no integration. The Core REST API instance is needed for tests 8 and 9, where the Script Runner reads endpoint groups through it.

TEST 1

Connectivity and authorisation

PACK CONTENT

- KOI integration (from the Marketplace KOI pack)

TENANT SETUP

- Marketplace KOI pack installed (it holds the integration)
- A KOI API key from the KOI console

Steps

- 1 Settings → Data Sources → Add an instance → search KOI.
- 2 **Server URL:** `https://api.prod.koi.security/`
- 3 Paste the API key, click Test, then Save.
- 4 Open the CLI on any incident and run:
- 5 `!koi-inventory-list limit=5`

What to expect

- ✓ Test returns Success.
- ✓ The command returns a table of inventory items.

TEST 2

Event collection

PACK CONTENT

- KOI integration
- KoiContentExtension Parsing Rule

TENANT SETUP

- Test 1 passing
- Fetch events enabled on the KOI instance

Steps

- 1 Edit the KOI instance and tick Fetch events, then Save.
- 2 Wait for two collection cycles.
- 3 Open Query Builder and run:
- 4 `dataset = koi_koi_raw`
- 5 `| comp count() as rows by source_log_type`

What to expect

- ✓ Rows appear under Alerts, Audit, or both.
- ✓ Counts grow between cycles.
- ✓ Fields such as `item_id` and `marketplace` are populated — this pack's parsing rule does that.

TEST 3

Counting alerts correctly

PACK CONTENT

- KoiContentExtension Parsing Rule
- KOI alerts dashboard

TENANT SETUP

- Test 2 passing — events present in koi_koi_raw

Steps

- 1 In Query Builder run:
- 2 `dataset = koi_koi_raw`
- 3 `| filter source_log_type = "Alerts"`
- 4 `| comp count() as rows, count_distinct(koi_notification_id) as real_alerts`
- 5 Then open the KOI Alerts Dashboard and compare.

What to expect

- ✓ rows is far larger than real_alerts.
- ✓ Dashboard alert counts match real_alerts, not rows.
- ✓ Counts stay stable as fetches repeat.

TEST 4

Alert triage

PACK CONTENT

- KOI Ext IR - Alert Triage
- KOI Ext IR - Extract Alert Context
- 19 KOI incident fields + incident type

TENANT SETUP

- Test 1 passing
- A KOI alert in XSIAM

Steps

- 1 Open a KOI alert.
- 2 Attach the triage playbook, or run:
- 3 `!setPlaybook playbookId="KOI Ext IR - Alert Triage"`
- 4 Watch the Work Plan, then read the War Room.

What to expect

- ✓ Context is extracted from the alert payload.
- ✓ A verdict is posted: Malicious, Suspicious or Benign.
- ✓ Low risk auto-closes; critical and high escalate.
- ✓ The 19 KOI alert fields are written onto the alert.

TEST 5

Item and device investigation

PACK CONTENT

- KOI Ext IR - Investigate Item
- KOI Ext IR - Investigate Device
- KOI Ext IR - Enrich Item

TENANT SETUP

- Test 1 passing
- An item id and its marketplace, and a hostname
- Nothing else — the XQL engine XSIAM uses for execution evidence is built in

Steps

- 1 Automation → Playbooks → KOI Ext IR - Investigate Item → Run.
- 2 Supply item_id and marketplace.
- 3 Repeat with KOI Ext IR - Investigate Device and a hostname.

What to expect

- ✓ An investigation summary with organisation exposure.
- ✓ Both governance counts — on blocklist AND on allowlist.
- ✓ The device view lists what KOI inventoried on that host.
- ✓ With Cortex XDR present, execution evidence is added.

TEST 6

Gated response

PACK CONTENT

- KOI Ext IR - Block and Remediate
- KOI Ext IR - Investigate Item

TENANT SETUP

- Test 1 passing
- An item id and marketplace
- Permission to approve a task

Steps

- 1 Automation → Playbooks → KOI Ext IR - Block and Remediate → Run.
- 2 Supply item_id and marketplace.
- 3 Open the pending approval task and read it.
- 4 Approve, or leave it pending.

What to expect

- ✓ The run PARKS on an approval task and waits.
- ✓ The approval shows the investigation and governance state.
- ✓ Nothing reaches the blocklist until a human approves.
- ✓ An item already blocked short-circuits.

TEST 7

Proactive hunting

PACK CONTENT

- KOI Ext Hunting - MCP Server Audit
- KOI Ext Hunting - Hunt Sweep
- KOI Ext Hunting - Hunt Match Investigation

TENANT SETUP

- Test 1 passing
- Nothing else — the hunt sweep queries xdr_data, which XSIAM holds natively

Steps

- 1 Automation → Playbooks → KOI Ext Hunting - MCP Server Audit → Run.
- 2 Then run KOI Ext Hunting - Hunt Sweep.
- 3 Optionally set hunt_set and min_risk.
- 4 Read the War Room match table.

What to expect

- ✓ The audit lists MCP servers at or above the threshold.
- ✓ The sweep runs its hunts and investigates what it finds.
- ✓ Block candidates are routed to an analyst gate, never blocked automatically.

Script Runner: everything this use case needs

PACK CONTENT — all of it

Five playbooks — import child before parent:

- 1 KOI Ext Script Runner - Execute Endpoint Script
- 2 KOI Ext Script Runner - Process Config Entry
- 3 KOI Ext Script Runner - Refresh Entry
- 4 KOI Ext Script Runner - Refresh Job
- 5 KOI Ext Script Runner - Scan Job

Plus:

KoiScanTracker — an automation, not a playbook. Ours, and different from the KOI deployment script opposite. For manual upload use dist/automation-KoiScanTracker.yml (one file).

TENANT SETUP

- ✓ Core REST API integration instance — the Refresh job reads endpoint groups through it.
- ✓ The KOI deployment script in Action Center → Scripts Library. Download it from YOUR KOI console; it is KOI's script, not a Cortex one, and must take no parameters.
- ✓ An endpoint group — tag the agents, then build a dynamic group on that tag.

Do this once, before tests 8 and 9

- 1 Installed the whole pack? All of the above is already on the tenant — go straight to test 8.
- 2 Uploading selectively? Automation → Playbooks → Import for the five playbooks, in the order listed.
- 3 Automation → Scripts → Import for KoiScanTracker — upload dist/automation-KoiScanTracker.yml. One file, code already inside; do not upload the two files under Scripts/.
- 4 Settings → Object Setup → Lists → New List, type JSON, named exactly Koi Script Runner. Paste the block from Lists/README.md.
- 5 Edit two things in that List: endpoint_groups and script.name. One entry per OS; the rest has working defaults.

Script Runner — refresh the tracker

PACK CONTENT

- KOI Ext Script Runner - Refresh Job
- KOI Ext Script Runner - Refresh Entry
- KoiScanTracker (automation)
- List: Koi Script Runner

TENANT SETUP

- Everything on the previous slide is in place

Steps

- 1 Automation → Jobs → New Job, time-triggered, playbook KOI Ext Script Runner - Refresh Job.
- 2 Enable the Job, then use Run now.

What to expect

- ✓ A tracker List appears under Lists, named as in your entry.
- ✓ It fills with one row per endpoint: an id and a last-scan value.
- ✓ Re-running adds new endpoints without disturbing existing rows.

Script Runner — scan due endpoints

PACK CONTENT

- KOI Ext Script Runner - Scan Job
- KOI Ext Script Runner - Process Config Entry
- KOI Ext Script Runner - Execute Endpoint Script
- KoiScanTracker (our automation)

TENANT SETUP

- Test 8 passing — the tracker List has rows
- Connected agents in the group, matching the entry's endpoint_os

Steps

- 1 Automation → Jobs → New Job → time-triggered → playbook KOI Ext Script Runner - Scan Job.
- 2 Enable the Job, then Run now.
- 3 Open the run, then check Action Center.
- 4 Run it a second time immediately.

What to expect

- ✓ The script is dispatched to due, connected endpoints.
- ✓ Those endpoints get a last-scan timestamp in the tracker.
- ✓ Offline and wrong-OS endpoints stay due and retry later.
- ✓ The second run does nothing — all inside the rescan interval.
- ✓ A SKIPPED entry means nothing was due; a STILL RUNNING entry means Action Center is finishing on its own. Both are normal and send no email.

Alert fields and layout

PACK CONTENT

- 19 KOI incident fields
- KOI Supply Chain Alert incident type
- its layout

TENANT SETUP

- Test 4 passing — triage has run on an alert
- The incident type and layout installed with this pack

Steps

- 1 Open an alert that triage has processed.
- 2 Check its type is KOI Supply Chain Alert.
- 3 Read the KOI Alert tab.

What to expect

- ✓ 19 KOI fields are populated on the alert itself.
- ✓ They are grouped: item, risk and verdict, affected host, governance, summary.
- ✓ An analyst sees the picture without opening the War Room.

One line of evidence per test

1

Test returns Success; inventory returns rows

2

koi_koi_raw grows; promoted fields are populated

3

Distinct alerts far below raw rows; dashboard matches

4

A verdict is reached; low risk auto-closes

5

Investigation shows exposure and both governance counts

6

Response parks on approval; blocklist untouched

7

MCP audit returns servers; hunt sweep completes

8

Tracker List is created and fills with endpoints

9

Scan marks only due, connected endpoints

10

19 KOI fields populated on the alert layout

All ten green — the extension is ready for production use.